Drew University - CSCI 260 - Fall 2024
In-Class Exercise 5 - C Pointers

Instruction: Please work in groups of 4 and write down your names for attendance. 12/02/2024
Names:

1. Without running the following program, identify its output. If line 4 is "int x = 7" instead, what is the output then?

```
int x;          // I am a global variable
int main() {
    int *px = &x;
    x = 7;
    *px = 4;
    printf("%d\n", x);
    return 0;
}
```

Ans: Since px is now the address of x,
*px is pointing towards the value of x.
Thus, *px = 4 overwrites the value of x as 4.
The printed result is then 4.

Whereas, if it was "int x = 7", then a local var is created. However, px is still the address of the global var x. Thus, changing *px won't affect the value of the local var x.
Therefore, the printed result is 7.

2. Without running the following program, identify its output.

```
void func(int* ptr) {
    *ptr = 30;
}
int main() {
    int val = 10;
    func(&val);
    printf("%d\n", val);
    return 0;
}
```

Ans: Since the address of val is passed into func and the pointer var ptr is receiving that address, ptr is now pointing towards the value of val.
Thus, by setting *ptr to 30, the printed result is going to be 30.

3. Without running the following program, identify its output.

```
int main() {
    int x;
    int *ptr = &x;
    *ptr = 5;
    printf("x = %d, *ptr = %d\n", x, *ptr);
    *ptr += 3;
    printf("x = %d, *ptr = %d\n", x, *ptr);
    (*ptr)++;
    printf("x = %d, *ptr = %d\n", x, *ptr);
    return 0;
}
```

Ans: First, *ptr is pointing towards value of x.
Thus, by setting *ptr to 5, both x and *ptr become 5.

Then, *ptr is added by 3 and the sum is assigned back to that address. Hence, both x and *ptr becomes 8.

Lastly, ++ increments the value it is applied on and implicitly assigns it back to the var where the value was taken. Here *ptr is taken and assigned back.
Therrfore, now both vars are holding 9.

4. The following function of converting letters in a string to lower-cases has some issues. In fact, if you try to print the returned string, nothing will be printed out. The issue is with the return type not being a char*. Run the program and try to fix it.

Ans: Simply change the declaration of newStr to a char pointer type would solve this; i.e., the first line within the function becomes: "char* newStr = (char*) malloc(sizeof(char)*MAX_LEN);".

```
#define MAX_LEN 20
char* toLowerCase(char* str) {
    char newStr[MAX_LEN];
    int i;
    for (i = 0; i < MAX_LEN; i++) {
        if ('A' <= str[i] && str[i] <= 'Z')
            newStr[i] = str[i] + ('a' - 'A');
        else
            newStr[i] = str[i];
    }
    return newStr;
}
```

5. Recall when swapping 2 numbers using a *swap_ptr()* function, we passed the addresses of the numbers as arguments and switched what the pointers were pointing towards in the function.

   Although it seems that the following *swap_string()* function for swapping 2 strings utilizes the same mechanism as *swap_ptr()*, it is NOT working as intended.

   Run the program and try to identify the problem (i.e., what makes the passing in this function different from that in *swap_ptr()?*) and then fix it.

Ans: This is equivalent to the error when NOT passing pointers of the number/char variables into the callee function, and the swap is then only within the scope of the function.

Although here, we are passing addresses, but they are addresses only because we are dealing with strings; i.e., char pointers. Here, we can think of char* as the equivalence of int. Therefore, instead, we need to deal with the addresses of those char* vars.

```
void swap_strings(char* s1, char* s2) {
    char* temp = s1;
    s1 = s2;
    s2 = temp;
}
int main() {
    char *s1 = "Hello";
    char *s2 = "World";
    swap_strings(s1, s2);
    return 0;
}
```

To fix this then, we need to pass in &s1 and &s2 instead in the main function.

Whereas, to receive those addresses, the arguments of the function becomes two pointers of strings (which themselves are pointer of chars); i.e., swap_strings(char** s1_addr, char** s2_addr).

Now, within the function, we should dereference the arguments to get the actual strings; i.e.,

```
char* temp = *s1_addr;
*s1_addr = *s2_addr;
*s2_addr = temp;
```

This will now swap the two string values within the memory.